

CSS125 - FA1.1

QUESTION 1.8

```
1  // ! This C++ program is written in C++23 !
2
3  #include <print>
4  #include <stdexcept>
5
6  namespace
7  {
8      constexpr auto factorial_of(unsigned int number) → unsigned long long
9      {
10         // Base case: factorial of 0 and 1 is 1
11         if ( number ≤ 1 ) {
12             return 1;
13         }
14
15         // Check for potential overflow (factorial grows very quickly! :D)
16         if ( constexpr auto overflow_limit{ 20 }; number > overflow_limit ) {
17             throw std::overflow_error("Factorial too large for unsigned long long");
18         }
19
20         // Recursive case: n! = n * (n-1)!
21         return number * factorial_of(number - 1);
22     }
23
24 } // namespace
25
26 auto main() → int
27 {
28
29     // This program prints the factorial of 10.
30     constexpr auto sample_number{ 10 };
31
32     try {
33         // Test the factorial function
34         for ( unsigned int counter{ 0 }; counter ≤ sample_number; ++counter ) {
35             std::println("{}! = {}", counter, factorial_of(counter));
36         }
37     } catch ( const std::exception& e ) {
38         std::println("Error: {}", e.what());
39         return 1;
40     }
41
42     return 0;
43 }
44
```

QUESTION 1.11

1. Abstraction And Information Hiding

Procedures allow us to hide implementation details behind a well-defined interface. The caller only needs to know *what* the function does (its contract), not *how* it accomplishes the task. This reduces cognitive load and allows programmers to think at higher levels of abstraction.

Example

The `factorial_of()` function hides the recursive implementation details. Users only need to know it takes an unsigned integer and returns the factorial value.

2. Code Reusability and DRY Principle

By packaging common operations into functions, we avoid duplicating code throughout our program (Don't Repeat Yourself). This makes our codebase more maintainable and reduces the chance of introducing bugs through copy-paste errors.

Example

Instead of writing the factorial calculation logic multiple times, we can call `factorial_of(n)` wherever needed.

3. Modularity And Testing

Functions create discrete, testable units that can be developed, debugged, and verified independently. This modular approach makes complex programs more manageable by breaking them into smaller, understandable components.

Example

Testing the `factorial_of()` function in isolation with various inputs (0, 1, 5, 20) to ensure it behaves correctly before integrating it into larger programs.